

Министерство образования Республики Беларусь

Учреждение образования
“Белорусский государственный университет
информатики и радиоэлектроники”

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2

по дисциплине «Логические основы интеллектуальных систем»

на тему

«Логическое программирование поиска решения задачи»

Вариант 4

Выполнил студент гр. 321701

Перминова В. В.

Проверил

Ивашенко В. П.

Минск 2025

Цель: Приобрести навыки логического программирования поиска решения задачи.

Постановка задачи: Два берега реки. На одном из них полицейский с заключённым, мама с дочерьми и отец с сыновьями. Необходимо с помощью плота, вмещающего не более двух человек, переправить всех персонажей на другой берег реки. Управлять плотом могут только полицейский и родители. Заключённого нельзя оставлять ни с одним из членов семьи. Папе не разрешается находиться с дочерьми без присутствия матери. Маме не разрешается находиться с сыновьями без присутствия отца

Уточнение постановки задачи: в связи с тем, что задачу невозможно решить для произвольного количества детей, было решено уточнить то, что в семье две дочери и два сына.

Дополнительные теоретические сведения:

Грамматика языка PROLOG.

$\langle \text{ПРОЛОГ-предложение} \rangle ::= \langle \text{правило} \rangle \mid \langle \text{факт} \rangle \mid \langle \text{запрос} \rangle$

$\langle \text{правило} \rangle ::= \langle \text{заголовок} \rangle \text{ ‘:-’ } \langle \text{тело} \rangle$

$\langle \text{факт} \rangle ::= \langle \text{заголовок} \rangle \text{ ‘.’}$

$\langle \text{запрос} \rangle ::= \langle \text{тело} \rangle \text{ ‘.’}$

$\langle \text{тело} \rangle ::= \langle \text{цель} \rangle \text{ ‘,’ } \langle \text{цель} \rangle \text{ ‘.’}$

$\langle \text{заголовок} \rangle ::= \langle \text{предикат} \rangle$

$\langle \text{цель} \rangle ::= \langle \text{предикат} \rangle \mid \langle \text{выражение} \rangle$

$\langle \text{предикат} \rangle ::= \langle \text{имя} \rangle \text{ / ‘(’ } \langle \text{терм} \rangle \text{ / ‘,’ } \langle \text{терм} \rangle \text{ / ‘)’ /}$

$\langle \text{терм} \rangle ::= \langle \text{атом} \rangle \mid \langle \text{предикат} \rangle \mid \langle \text{список} \rangle$

$\langle \text{атом} \rangle ::= \langle \text{переменная} \rangle \mid \langle \text{число} \rangle \mid \langle \text{строка} \rangle \mid \langle \text{имя} \rangle$

$\langle \text{список} \rangle ::= \langle \text{список с заголовком} \rangle \mid \langle \text{простой список} \rangle$

$\langle \text{список с заголовком} \rangle ::= \text{ ‘[’ } \langle \text{терм} \rangle \text{ / ‘,’ } \langle \text{терм} \rangle \text{ / ‘|’ } \langle \text{терм} \rangle \text{ ’} \text{’}$

$\langle \text{простой список} \rangle ::= \text{ ‘[’ } \langle \text{терм} \rangle \text{ / ‘,’ } \langle \text{терм} \rangle \text{ / ‘|’ } \text{ ‘[’ } \text{’}$

$\langle \text{выражение} \rangle ::= \langle \text{терм} \rangle \text{ / } \langle \text{оператор} \rangle \langle \text{терм} \rangle \text{ /}$

$\langle \text{оператор} \rangle ::= \text{ ‘is’ } \mid \text{ ‘!’ } \mid \text{ ‘==’ } \mid \text{ ‘\=’ } \mid \text{ ‘>=’ } \mid \text{ ‘<=’ } \mid \text{ ‘=\=’ } \mid$

Для решения данной задачи был использован ряд встроенных правил:

`format(+Format, +Arguments)` – выводит форматированную строку. `Format` — строка управления, `Arguments` — список подставляемых значений.

`sort(+List, -Sorted)` – сортирует список `List` в порядке возрастания, удаляет дубликаты и возвращает результат в `Sorted`.

`member(?Elem, ?List)` – проверяет, что `Elem` является элементом списка `List`, или генерирует элементы списка при переборе.

`select(?Element, ?List, ?Rest)` – выбирает элемент из списка `List` и возвращает оставшиеся элементы в список `Rest`.

`var(@Term)` – проверяет, является ли переданный ему аргумент `Term` неопределённой переменной (т.е. "свободной", без значения). Возвращает `true`, если `Term` — свободная переменная.

Использованные встроенные операторы

`= (?Term1, ?Term2)` – проверяет идентичность термов или присваивает значения термам (унификация).

`(If -> Then ; Else)` – условное выполнение. Если `If` истинен, выполняется `Then`; иначе выполняется `Else`.

`[Head | Tail]` – используется для разделения списка на голову `Head` и хвост `Tail`. Голова списка представляет собой первый элемент, а хвост — это оставшаяся часть списка.

Логические связки:

1. `,` - обозначение конъюнкции;
2. `;` - обозначение дизъюнкции;
3. `:-` - обозначение импликации;
4. `\+` - обозначение отрицания.

Реализованные факты:

1. `family([mother, father, daughter1, daughter2, son1, son2])`. – список членов семьи.
2. `daughters([daughter1, daughter2])`. – список дочерей.
3. `sons([son1, son2])`. – список сыновей.

4. `can_steer_list([policeman, mother, father])`. – список тех, кто может управлять ПЛОТОМ.
5. `print_solution([])`. – пустой путь.

Реализованные правила:

1. $\forall \text{Person } \forall \text{Steerers } ((\text{can_steer_list}(\text{Steerers}) \wedge \text{member}(\text{Person}, \text{list}(\text{Steerers}))) \rightarrow \text{can_steer}(\text{Person})).$
2. $\forall \text{Bank } \forall F \forall M \forall D \forall \text{Daughter } \forall S \forall \text{Son } ((!(\text{member}(\text{prisoner}, \text{Bank}) \wedge (\text{family}(F) \wedge \text{member}(M, F) \wedge \text{member}(M, \text{Bank}))) \wedge !(\text{member}(\text{policeman}, \text{Bank})))) \wedge !(\text{member}(\text{father}, \text{Bank}) \wedge (\text{daughters}(D) \wedge \text{member}(\text{Daughter}, D) \wedge \text{member}(\text{Daughter}, \text{Bank})) \wedge !(\text{member}(\text{mother}, \text{Bank})))) \wedge !(\text{member}(\text{mother}, \text{Bank}) \wedge (\text{sons}(S) \wedge \text{member}(\text{Son}, S) \wedge \text{member}(\text{Son}, \text{Bank})) \wedge !(\text{member}(\text{father}, \text{Bank})))))) \rightarrow \text{safe_bank}(\text{Bank})).$
3. $\forall \text{SLeft } \forall \text{Left } \forall \text{SRight } \forall \text{Right } \forall \text{Boat } ((\text{sort}(\text{Left}, \text{SLeft}) \wedge \text{sort}(\text{Right}, \text{SRight})) \rightarrow \text{normalize_state}(\text{list}(\text{Left}, \text{Right}, \text{Boat}), \text{list}(\text{SLeft}, \text{SRight}, \text{Boat}))).$
4. $\forall \text{Steerer } \forall \text{Left } \forall \text{TempLeft } \forall \text{Passenger } \forall \text{Right } \forall \text{NewLeft } \forall \text{NewRight } ((\text{select}(\text{Steerer}, \text{Left}, \text{TempLeft}) \wedge \text{can_steer}(\text{Steerer}) \wedge (\text{select}(\text{Passenger}, \text{TempLeft}, \text{NewLeft}) \vee \text{NewLeft} \sim \text{TempLeft}) \wedge ((\text{var}(\text{Passenger}) \rightarrow (\text{NewRight} \sim \text{list}(\text{Steerer}|\text{Right}))) \vee (\text{NewRight} \sim \text{list}(\text{Steerer}, \text{Passenger}|\text{Right})))) \wedge \text{safe_bank}(\text{NewLeft}) \wedge \text{safe_bank}(\text{NewRight})) \rightarrow \text{move}(\text{list}(\text{Left}, \text{Right}, \text{left}), \text{list}(\text{NewLeft}, \text{NewRight}, \text{right}))).$
5. $\forall \text{Steerer } \forall \text{Left } \forall \text{TempRight } \forall \text{Passenger } \forall \text{Right } \forall \text{NewLeft } \forall \text{NewRight } ((\text{select}(\text{Steerer}, \text{Right}, \text{TempRight}) \wedge \text{can_steer}(\text{Steerer}) \wedge (\text{select}(\text{Passenger}, \text{TempRight}, \text{NewRight}) \vee (\text{NewRight} \sim \text{TempRight})) \wedge ((\text{var}(\text{Passenger}) \rightarrow (\text{NewLeft} \sim \text{list}(\text{Steerer}|\text{Left}))) \vee (\text{NewLeft} \sim \text{list}(\text{Steerer}, \text{Passenger}|\text{Left})))) \wedge \text{safe_bank}(\text{NewLeft}) \wedge \text{safe_bank}(\text{NewRight})) \rightarrow \text{move}(\text{list}(\text{Left}, \text{Right}, \text{right}), \text{list}(\text{NewLeft}, \text{NewRight}, \text{left}))).$
6. $\forall \text{State } \forall \text{Right } \forall \text{Sorted } ((\text{normalize_state}(\text{State}, \text{list}(\text{list}(), \text{Right}, _)) \wedge \text{sort}(\text{Right}, \text{Sorted}) \wedge \text{Sorted} \sim \text{list}(\text{daughter1}, \text{daughter2}, \text{father}, \text{mother}, \text{policeman}, \text{prisoner}, \text{son1}, \text{son2})) \rightarrow \text{solve}(\text{State}, \text{list}(\text{State}), _)).$
7. $\forall \text{State } \forall \text{Path } \forall \text{Visited } \forall \text{NextState } \forall \text{NormNext } ((\text{move}(\text{State}, \text{NextState}) \wedge \text{normalize_state}(\text{NextState}, \text{NormNext}) \wedge !(\text{member}(\text{NormNext}, \text{Visited})) \wedge \text{solve}(\text{NextState}, \text{Path}, \text{list}(\text{NormNext}|\text{Visited}))) \rightarrow \text{solve}(\text{State}, \text{list}(\text{State}|\text{Path}), \text{Visited})).$
8. $\forall \text{NormInit } \forall \text{Path } ((\text{InitialState} \sim \text{list}(\text{list}(\text{policeman}, \text{prisoner}, \text{mother}, \text{father}, \text{daughter1}, \text{daughter2}, \text{son1}, \text{son2}), \text{list}(), \text{left}) \wedge \text{normalize_state}(\text{InitialState}, \text{NormInit}) \wedge \text{solve}(\text{InitialState}, \text{Path}, \text{list}(\text{NormInit})) \wedge \text{print_path}(\text{Path})) \rightarrow \text{start}()).$

9. $\forall \text{State} \forall \text{Rest} ((\text{State} \sim \text{list}(\text{Left}, \text{Right}, \text{Boat}) \wedge \text{format}(\text{'Left: } \sim w, \text{Right: } \sim w, \text{Boat: } \sim w \sim n', \text{list}(\text{Left}, \text{Right}, \text{Boat})) \wedge \text{print_path}(\text{Rest})) \rightarrow \text{print_path}(\text{list}(\text{State} | \text{Rest})))$

Код программы:

% Лабораторная работа №2 по дисциплине Логические основы интеллектуальных систем

% Выполнена студентом группы 321701 БГУИР Перминовой Викторией Вячеславовной

% В файле содержится исходный код для решения задачи: "Два берега реки. На одном из них полицейский с заключённым, мама с дочерьми и отец с сыновьями. Необходимо с помощью плота, вмещающего не более двух человек, переправить всех персонажей на другой берег реки. Управлять плотом могут только полицейский и родители. Заключённого нельзя оставлять ни с одним из членов семьи. Папе не разрешается находиться с дочерьми без присутствия матери. Маме не разрешается находиться с сыновьями без присутствия отца".

% 20.05.2025 1.0

% Списки членов семьи

family([mother, father, daughter1, daughter2, son1, son2]).

daughters([daughter1, daughter2]).

sons([son1, son2]).

% Список тех, кто может управлять плотом

can_steer_list([policeman, mother, father]).

% Предикат для проверки возможности управления

can_steer(Person) :-

can_steer_list(Steerers),

member(Person, Steerers).

% Безопасность берега

safe_bank(Bank) :-

```
\+ (member(prisoner, Bank),  
    (family(F), member(M, F), member(M, Bank)),  
    \+ member(policeman, Bank)),  
\+ (member(father, Bank),  
    (daughters(D), member(Daughter, D), member(Daughter, Bank)),  
    \+ member(mother, Bank)),  
\+ (member(mother, Bank),  
    (sons(S), member(Son, S), member(Son, Bank)),  
    \+ member(father, Bank)).
```

% Нормализация состояния

normalize_state([Left, Right, Boat], [SLeft, SRight, Boat]) :-

```
    sort(Left, SLeft),  
    sort(Right, SRight).
```

% Перемещение

move([Left, Right, left], [NewLeft, NewRight, right]) :-

```
    select(Steerer, Left, TempLeft),  
    can_steer(Steerer),  
    (select(Passenger, TempLeft, NewLeft) ; NewLeft = TempLeft),  
    ( var(Passenger)  
      -> NewRight = [Steerer | Right]  
      ; NewRight = [Steerer, Passenger | Right]  
    ),  
    safe_bank(NewLeft),  
    safe_bank(NewRight).
```

```

move([Left, Right, right], [NewLeft, NewRight, left]) :-
    select(Steerer, Right, TempRight),
    can_steer(Steerer),
    (select(Passenger, TempRight, NewRight) ; NewRight = TempRight),
    ( var(Passenger)
      -> NewLeft = [Steerer | Left]
      ; NewLeft = [Steerer, Passenger | Left]
    ),
    safe_bank(NewLeft),
    safe_bank(NewRight).

```

% Поиск решения

```

solve(State, [State], _) :-
    normalize_state(State, [], Right, _),
    sort(Right, Sorted),
    Sorted = [daughter1, daughter2, father, mother, policeman, prisoner, son1, son2].

```

```

solve(State, [State | Path], Visited) :-
    move(State, NextState),
    normalize_state(NextState, NormNext),
    \+ member(NormNext, Visited),
    solve(NextState, Path, [NormNext | Visited]).

```

% Запуск

```

start :-

```

```

    InitialState = [

```

```

    [policeman, prisoner, mother, father, daughter1, daughter2, son1, son2],
    [],
    left
],
normalize_state(InitialState, NormInit),
solve(InitialState, Path, [NormInit]),
print_path(Path).

```

% Вывод пути

```
print_path([]). % База рекурсии: пустой путь
```

```
print_path([State | Rest]) :-
```

```
    State = [Left, Right, Boat],
```

```
    format('Left: ~w, Right: ~w, Boat: ~w~n', [Left, Right, Boat]),
```

```
    print_path(Rest).
```


Дерево вывода:



Рис. 1 - Дерево вывода

Тестирование:

```
?- start.
Left: [policeman,prisoner,mother,father,daughter1,daughter2,son1,son2], Right: [], Boat: left
Left: [mother,father,daughter1,daughter2,son1,son2], Right: [policeman,prisoner], Boat: right
Left: [policeman,mother,father,daughter1,daughter2,son1,son2], Right: [prisoner], Boat: left
Left: [mother,father,daughter2,son1,son2], Right: [policeman,daughter1,prisoner], Boat: right
Left: [policeman,prisoner,mother,father,daughter2,son1,son2], Right: [daughter1], Boat: left
Left: [policeman,prisoner,father,son1,son2], Right: [mother,daughter2,daughter1], Boat: right
Left: [mother,policeman,prisoner,father,son1,son2], Right: [daughter2,daughter1], Boat: left
Left: [policeman,prisoner,son1,son2], Right: [mother,father,daughter2,daughter1], Boat: right
Left: [father,policeman,prisoner,son1,son2], Right: [mother,daughter2,daughter1], Boat: left
Left: [father,son1,son2], Right: [policeman,prisoner,mother,daughter2,daughter1], Boat: right
Left: [mother,father,son1,son2], Right: [policeman,prisoner,daughter2,daughter1], Boat: left
Left: [son1,son2], Right: [mother,father,policeman,prisoner,daughter2,daughter1], Boat: right
Left: [father,son1,son2], Right: [mother,policeman,prisoner,daughter2,daughter1], Boat: left
Left: [son2], Right: [father,son1,mother,policeman,prisoner,daughter2,daughter1], Boat: right
Left: [policeman,prisoner,son2], Right: [father,son1,mother,daughter2,daughter1], Boat: left
Left: [prisoner], Right: [policeman,son2,father,son1,mother,daughter2,daughter1], Boat: right
Left: [policeman,prisoner], Right: [son2,father,son1,mother,daughter2,daughter1], Boat: left
Left: [], Right: [policeman,prisoner,son2,father,son1,mother,daughter2,daughter1], Boat: right
true
```

Рис. 2 – Результат работы программы

Вывод:

В ходе выполнения лабораторной работы были приобретены навыки логического программирования поиска решения задачи; были изучены основы языка программирования “Prolog”. С помощью среды программирования “SWI - Prolog” была разработана программа для решения логической задачи.

Список использованных источников:

1. Логические основы интеллектуальных систем. Практикум : учеб.-метод. пособие / В. В. Голенков [и др.]. – Минск : БГУИР, 2011. – 70 с. : ил. ISBN 978-985-488-487-5.
2. Пролог [Электронный ресурс]. – Режим доступа: <http://www.verim.org/project/prolog/>
3. Логическое программирование на Prolog [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/552318/>